

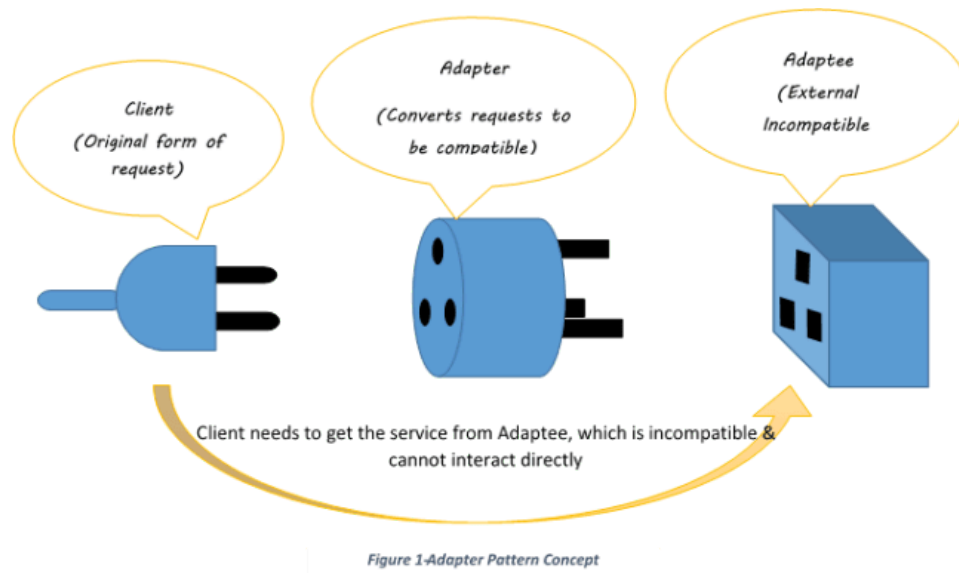
# Structural Patterns

## 1. Adapter Pattern ([https://www.youtube.com/watch?v=ZiIQ2Xp\\_urw](https://www.youtube.com/watch?v=ZiIQ2Xp_urw))

It is used to allow incompatible interfaces to work together. Acts as a bridge between two interfaces. ( Converting one interface to another as client expects. )

eg. Indian charger → US sockets (wont work)

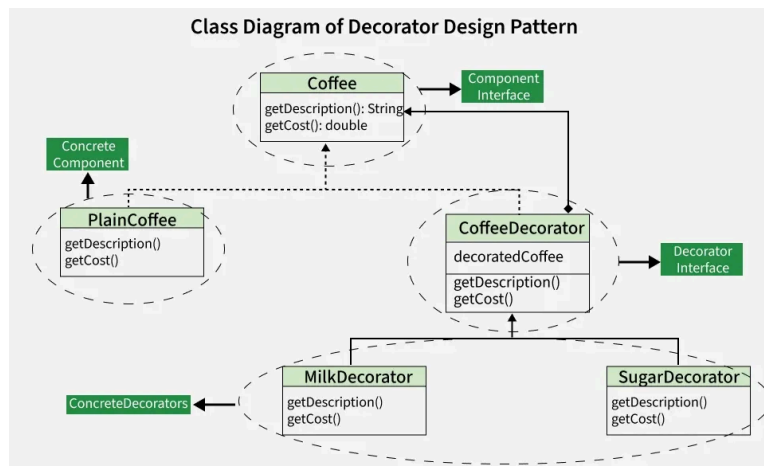
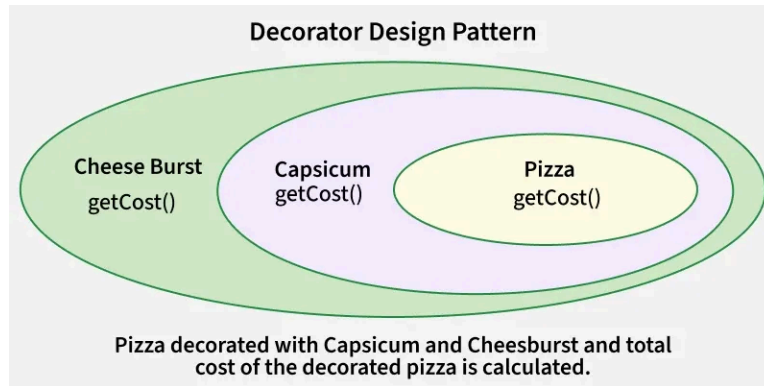
Indian Charger → Adapter (convert pins to US socket) → US socket



## 2. Decorator Pattern (<https://www.youtube.com/watch?v=XBnRL4MtR2U>)

It is used to dynamically add behavior or responsibilities to the individual objects at runtime, without modifying their code.

- i. It creates flexibility and reusability over subclassing.
- ii. Follows open/closed principle.



example: As there is a coffee.

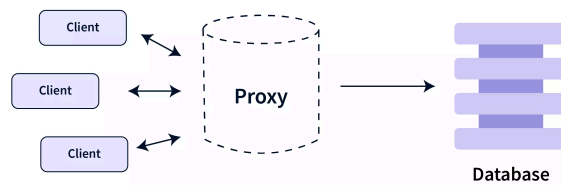
Coffee

1. Normal coffee
2. Chocolate coffee
3. cold coffee
4. coffee with less sugar

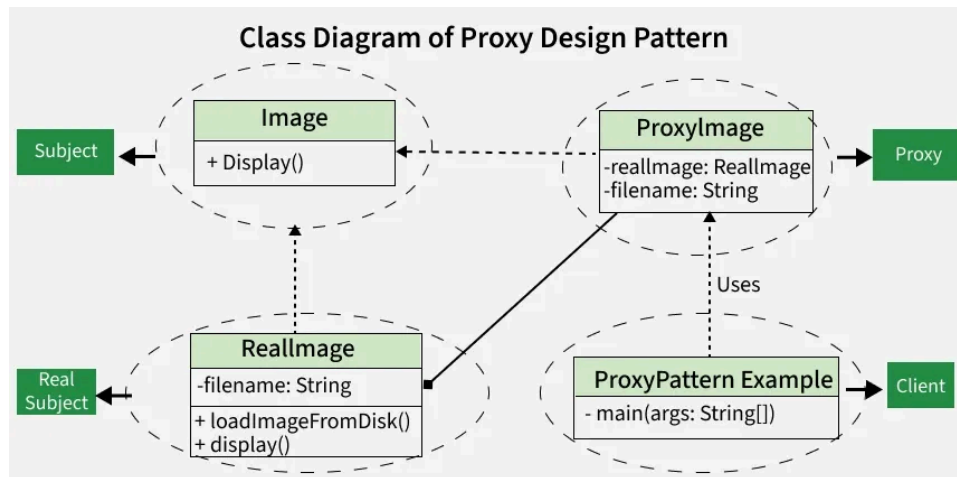
there is coffee but the behavior is changes to each.

### 3. Proxy Pattern

It provides a surrogate or placeholder object (Intermediate obj) to control access to the real object.



SCALER  
Topics



## Key Components

1. **Subject:** The interface shared by both the real object and the proxy. This ensures the proxy can be used wherever the real object is expected.
2. **Real Subject:** The actual, resource-intensive business object that performs the core logic.
3. **Proxy:** The placeholder object that implements the Subject interface, holds a reference to the Real Subject, and manages access to it.

The client interacts with the proxy, which acts as a middleman to perform additional tasks (like security, logging, or caching) **before forwarding requests** to the actual object